Spatial Data

Daria Agarkova, Filipa Reis, Stefan Salcher

This paper looks into Spatial Data Analysis, based on "Spatial Data Science" and "Spatial Economics", and applies a common data set for examplification. We conducted our analysis on R and Python.

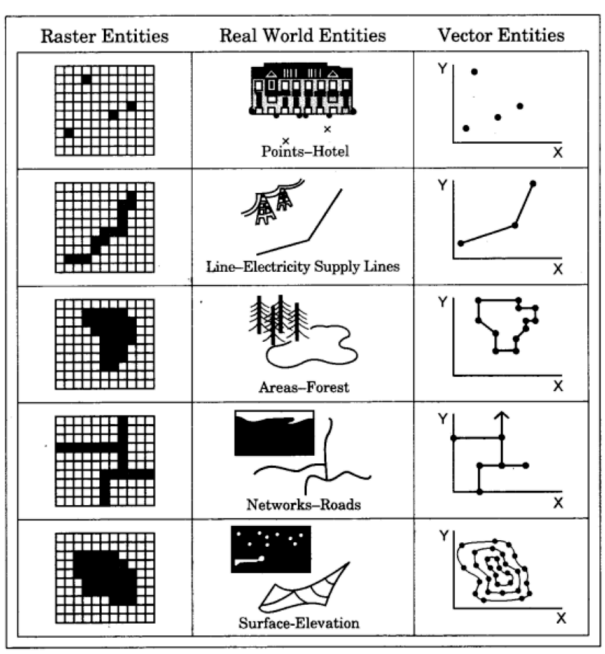
1 Spatial Data Logic and Intuition

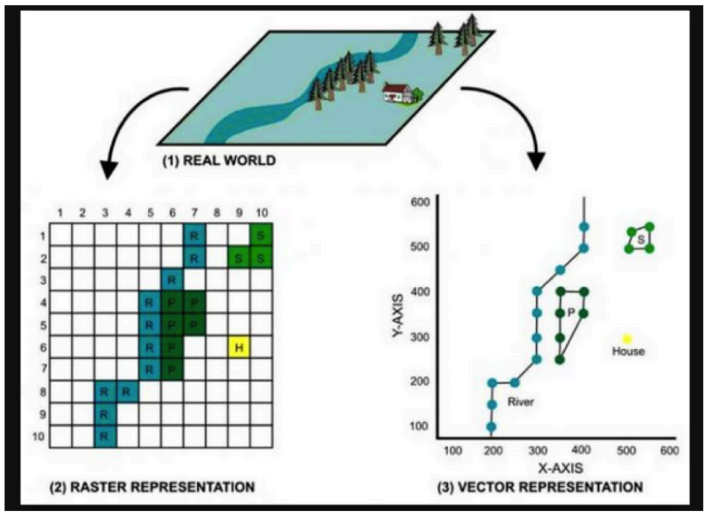
This section looks into the intuition and basic concepts of spatial data types and storage, in order to understand the operations of spatial analysis and how to load the data files.

1.1 Understanding Spatial Data

Spatial data refers to data about geographical locations, and can be general thought of as one of two things:

- Discrete objects with clear boundaries (**Vector Data**) : eg. country
- Continuous Phenomena without boundaries (**Raster Data**) : eg. temperature





1.1.1 Vector data

It is simply shapes i.e. **points, lines and polygons.** to represent the geometry of the real world - for all of them, the geometry consists of sets of coordinate pairs (x, y).

A **point** has one coordinate pair, and n associated variables.

- Eg: a point represents the coordinates of a person in a campsite, and attributes could include gender, height, date, etc.

Lines are represented as ordered sets of coordinates (nodes) so that an actual line segment can be constructed by connecting the points. A line is a set of 1 or more "polylines", i.e a connected series of line segments.

- Eg: in the campsite, there might be a river which has tributaries. The river with all its tributaries can be 1 line sigment, but each tributary can be it's own line segment.
- A network is a special type of line geometry that has additional info (eg: roads, river networks)

A **polygon** refers to a set of closed polylines - the geometry itself is very similar to that of lines, but to close a polygon the last coordinate pair is the same as the first pair.

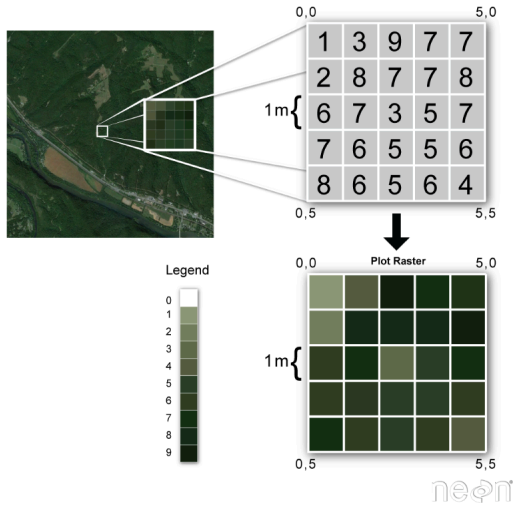
- Polygons can have holes (a polygon inside a polygon) - Eg: in the campsite there is also a lake, with an island inside it.
- Multiple polygons can be considered as a single geometry - Eg. a country with islands is represented with a single (multi-) polygon)

Vector data is best used with (but not exclusive to) discrete data (eg: buildings, roads, trees, locations, administrative boundaries)

1.1.2 Raster data

It's designed to store geospatial data information that is continuous across space, so it's the most common kind of data for (satellite imagery, climate variables).

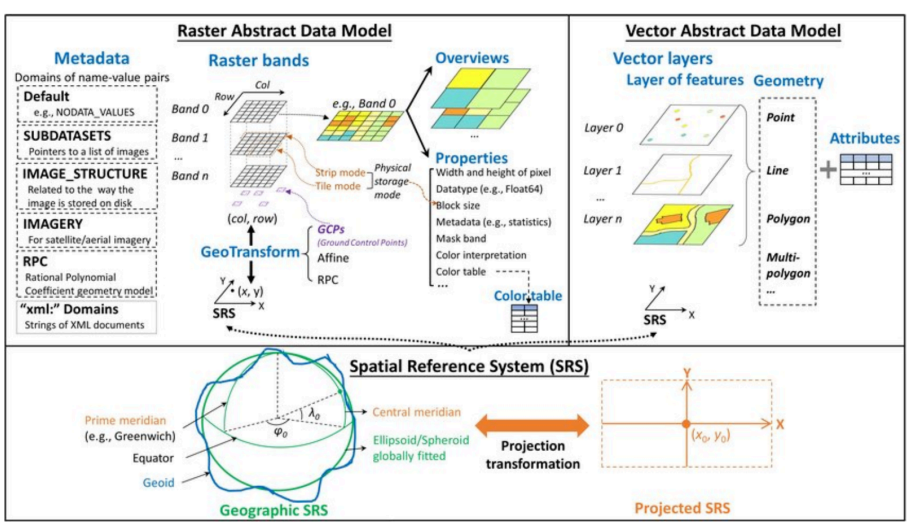
It's stored in a grid format -i.e. it divides the world into a grid of equally sized rectangles (pixels) that all have one or more values for the variables of interest, where each cell value should normally represent the average value for the area it covers.



In contrast to vector data, in raster data the geometry is not explicitly stored as coordinates. It is implicitly set by knowing the extent and the number or rows and columns in which the area is divided. In it's raw form it looks like a matrix.

Raster data is best used with continuous data (eg: temperature, soil moisture, elevation, air quality)

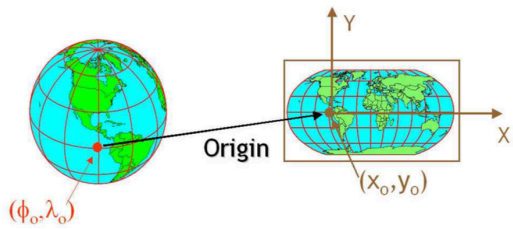
1.1.2.1 Overview Table



1.2 Projections & Coordinate Reference System (CRS)

Coordinate Reference System (CRS) - also called Spatial Reference System (SRS) - describes how coordinates relate to real-world location. It basically defines how latitude and longitude (X/Y coordinates) relate to real locations on Earth.

Go here an Overview of CRS in R.

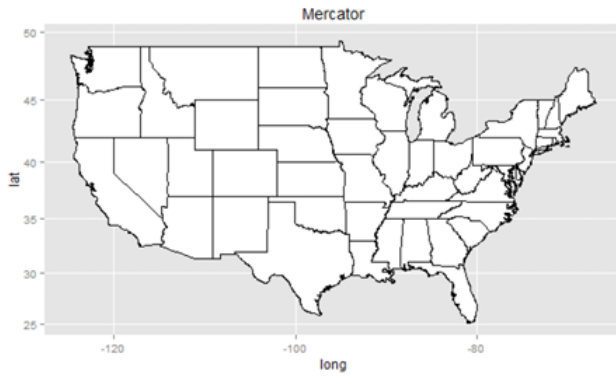


The CRS is made up of several key components:

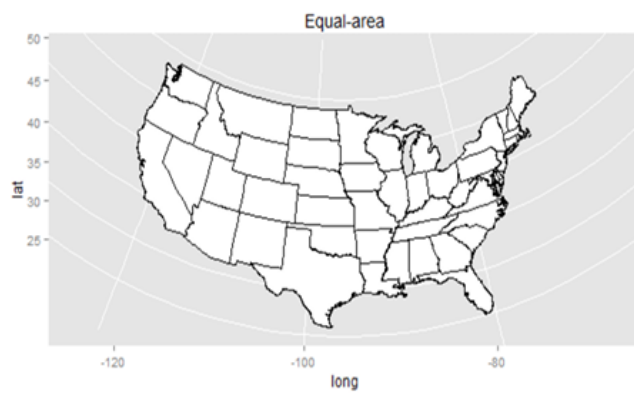
- **Coordinate system:** mathematical rules for specifying how coordinates (X,Y) are to be assigned to points
- **Horizontal and Vertical units:** the units used to define the grid along the x, y (and z) axis
- **Datum (geodetic CRS):** a modeled version of the shape of the Earth which defines the origin used to place the coordinate system in space.
  - The simplest datums are a sphere that is "flattened" at the poles and bulges at the equator. More complex datums allow for more variation in the earth's shape.
  - The most commonly used datum is called **WGS84** (World Geodesic System 1984), similar to NAD83 (The North American Datum of 1983). Other, local datums exist to more precisely record locations for a single country or region.
- **Projection Information:** The mathematical equation used to flatten objects that are on a round surface so you can view them on a flat surface
  - When the projection is done, you will always have some distortions, i.e. we can only preserve 1–2 of these 4 properties (this is a mathematical limitation):
    1. Area
    2. Shape
    3. Distance
    4. Direction
  - Examples are "Mercator", "UTM", "Robinson", "Lambert", "Sinusoidal" and "Albers".
  - List of most Common Projections: https://proj.org/en/stable/operations/projections/index.html
  - For more reference: https://spatialreference.org/

Example:

On the **Mercator projection** (used in Google Maps), Greenland looks as big as Africa, but in reality, Africa is 14 times larger. This is because Mercator keeps angles (shape) accurate, which is great for sailing, but it stretches size near the poles.



Mercator preserves direction and is useful for navigation. But distances and areas are distorted, especially near the polar regions



Azimuthal Equal Area preserves area, but not direction.

Most commonly used CRS have been assigned a "EPSG code" (European Petroleum Survey Group) - this is a unique ID that can be a simple way to identify a CRS.

1.2.1 Why is this important?

It is important to understand the coordinate system that your data uses, specially when working with different data sets as **data can be stored in different coordinate systems!**

Keep in mind :

If you use different data sets, you have to make sure they are all using the same projection!

To do this, you might have to transform the projection (using R):

```
# check the projection
st_crs(columbus)

# e.g. to transform to austria lambert
columbus_lambert <- st_transform(columbus, 31287)

# also notice how the coordinates change
columbus$geometry
columbus_lambert$geometry
```

Assume you have 2 spatial data sources (data_A and data_B), and you transform one (data_A) into the CRS of the other (data_B). Sometimes you will still encounter problems with some minor differences with overlapping.

You can fix this by:

```
#taking the difference in the centroid coordinates:m
st_centroid(st_union(data_A)); st_centroid(st_union(transform-data_B))

# and then shift the coordinates of the second dataset
raster::shift(transformed-data_B, dx, dy)
```

1.3 Data Types & Storage

There are many ways to store spatial data, 2 common formats are:

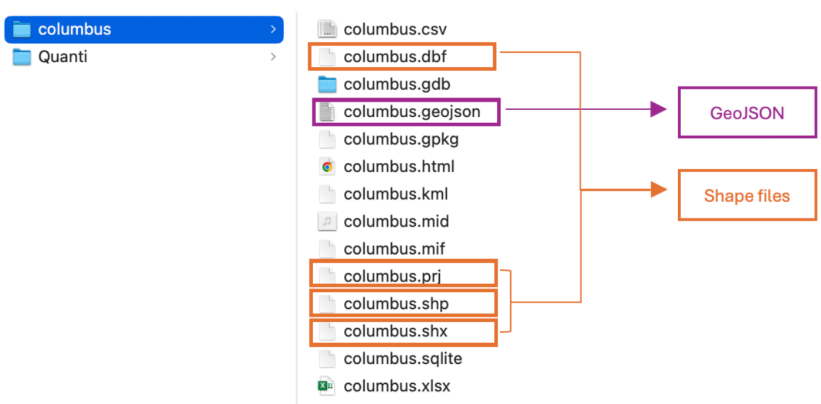
1. **GeoJSON** (single text file, human readable, good web support but limited crs support not ideal for large datasets);

- .geojson: a cleaner and more modern way to store data

2. **Shapefiles** (multiple files, non human readable, full crs support, small file size).

- .shp **:** a collection of files that all have the same name but different extensions (store metadata)
  - .shp : stores the geometry (points, lines, polygons)
  - .shx : an index file that maps features' positions in .shp (like a table of contents)
  - .dbf : stores attributes (tabular data linked to geometry)
  - .prj : the projectionand coordinate system

Example of our data file "columbus":



2 Our Dataset, Research Question & Variables

For convinience, we choose a classic dataset used in spatial econometrics, i.e. the Columbus Neighborhood Crime 1980 dataset. This dataset contains socioeconomic and crime indicators for 49 neighborhoods in Columbus, Ohio for the year 1980.

Variable	Description
AREA	Neighborhood area (computed by ArcView)
PERIMETER	Neighborhood perimeter (computed by ArcView)
COLUMBUS_	Internal polygon ID (generated by ArcView)
COLUMBUS_I	Internal polygon ID (geneated by ArcView)
POLYID	Neighborhood ID, used in GeoDa User's Guide and tutorials
NEIG	Neighborhood ID, used in Spatial Econometrics examples
HOVAL	Housing value (in $1,000)
INC	Household income (in $1,000)
CRIME	Residential burglaries and vehicle thefts per 1000 households
OPEN	Open space (area)
PLUMB	Percentage of housing units without plumbing
DISCBD	Distance to CBD
X	Centroid x coordinate (in arbitrary digitizing units)
Y	Centroid y coordinate (in arbitrary digitizing units)
NSA	North-south indicator variable (North = 1)
NSB	Other north-south indicator variable (North = 1)
EW	East-west indicator variable (East = 1)
CP	Core-periphery indicator variable (Core = 1)
THOUS	Constant (= 1000)
NEIGNO	Another neighborhood ID variable (NEIG + 1000)

Variables of Interest : "CRIME"; "HOVAL"; "INC"; "geometry".

Our Research Question is thus:

> **How do income levels and housing values affect local crime rate, and do nearby neighborhoods influence each other?**

3 Packages for Spatial Data

This section analyses Spatial Data with R based on "Spatial Data Science with R"

In R, working with vector and raster data involves distinct packages optimized for spatial operations, analysis, and visualization.

3.1 R Packages

```
options(repos = c(CRAN = "https://cloud.r-project.org"))

packages <- c("sf", "stars", "terra", "tidyverse", "dplyr",
              "ggplot2", "spatialreg", "spdep", "MASS",
              "paletteer", "ggthemes")
#sapply(packages, install.packages, character.only = T)
```

```r
sapply(packages, library, character.only = T)

# Use sf for all new vector work (sucessor to old sp package)
# Use terra for raster (and vector when you want one package)
# Use stars if you're working with time-series or multidimensional rasters

# spdep is for calcualting spatial autocorrelation

# tidyverse is the standard package collection for data wrangling
# paletteer & ggthemes are for colour palates for the graphs
```

3.2 Python Packages

This section analyses Spatial Data with Python - for more info on the Geopandas package go here.

```python
import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from libpysal.weights import KNN
from esda.moran import Moran
from spreg import OLS, ML_Lag
```

4 Spatial Data Analysis

4.1 Initial Data Wrangling

We begin our analysis by taking a look at the data and visually looking at it .

4.1.1 On R

Looking into the data:

```r
columbus <- st_read("SpatialData/columbus.shp")
```

An overview of what the data means:

```
> columbus
Simple feature collection with 49 features and 20 fields
Geometry type: POLYGON
Dimension:     XY
Bounding box:  xmin: 5.874907 ymin: 10.78863 xmax: 11.28742 ymax: 14.74245
Geodetic CRS:  WGS 84
First 10 features:
        AREA PERIMETER COLUMBUS_ COLUMBUS_I POLYID NEIG  HOVAL    INC     CRIME
1  0.309441  2.440629         2          5      1    5 80.467 19.531 15.725980
2  0.259329  2.236939         3          1      2    1 44.567 21.232 18.801754
3  0.192468  2.187547         4          6      3    6 26.350 15.956 30.626781
4  0.083841  1.427635         5          2      4    2 33.200  4.477 32.387760
5  0.488888  2.997133         6          7      5    7 23.225 11.252 50.731510
6  0.283079  2.335634         7          8      6    8 28.750 16.029 26.066658
7  0.257084  2.554577         8          4      7    4 75.000  8.438  0.178269
8  0.204954  2.139524         9          3      8    3 37.125 11.337 38.425858
9  0.500755  3.169707        10         18      9   18 52.600 17.586 30.515917
10 0.246689  2.087235        11         10     10   10 96.400 13.598 34.000835
       OPEN    PLUMB DISCBD     X     Y NSA NSB EW CP THOUS NEIGNO
1  2.850747 0.217155   5.03 38.80 44.07   1   1  1  0  1000   1005
2  5.296720 0.320581   4.27 35.62 42.38   1   1  0  0  1000   1001
3  4.534649 0.374404   3.89 39.82 41.18   1   1  1  0  1000   1006
4  0.394427 1.186944   3.70 36.50 40.52   1   1  0  0  1000   1002
5  0.405664 0.624596   2.83 40.01 38.00   1   1  1  0  1000   1007
6  0.563075 0.254130   3.78 43.75 39.28   1   1  1  0  1000   1008
7  0.000000 2.402402   2.74 33.36 38.41   1   1  0  0  1000   1004
8  3.483478 2.739726   2.89 36.71 38.71   1   1  0  0  1000   1003
9  0.527488 0.890736   3.17 43.44 35.92   1   1  1  0  1000   1018
10 1.548348 0.557724   4.33 47.61 36.42   1   1  1  0  1000   1010
                       geometry
1  POLYGON ((8.624129 14.23698...
2  POLYGON ((8.25279 14.23694,..
3  POLYGON ((8.653305 14.00809...
4  POLYGON ((8.459499 13.82035...
5  POLYGON ((8.685274 13.63952...
6  POLYGON ((9.401384 13.5504,...
7  POLYGON ((8.037741 13.60752...
8  POLYGON ((8.247527 13.58651...
9  POLYGON ((9.333297 13.27242...
10 POLYGON ((10.08251 13.03377...
```

objects/rows

atribute variables / columns

meta-data

geometry of the data

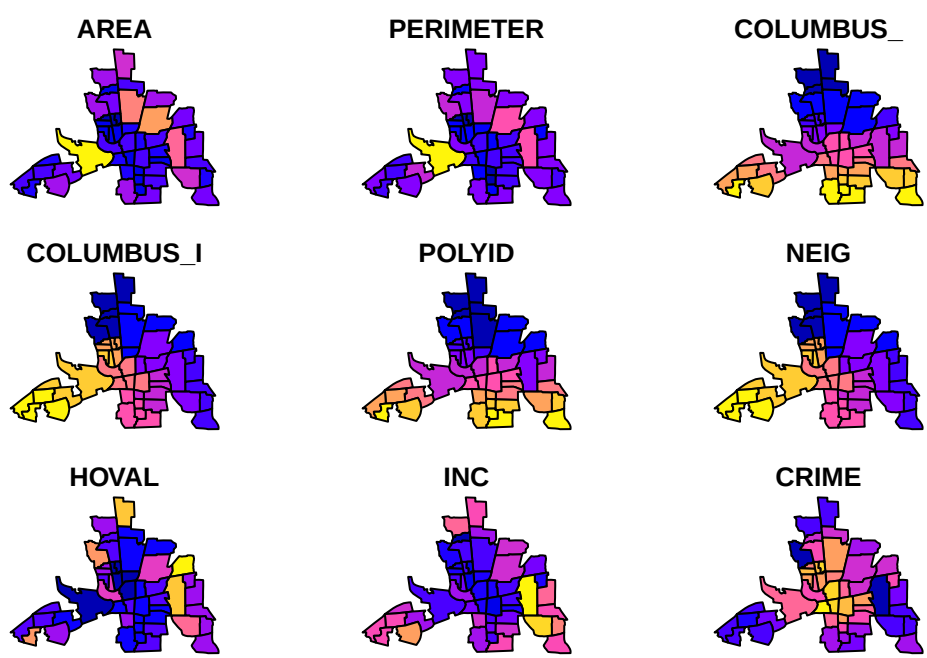
coordinate reference

simple feature set

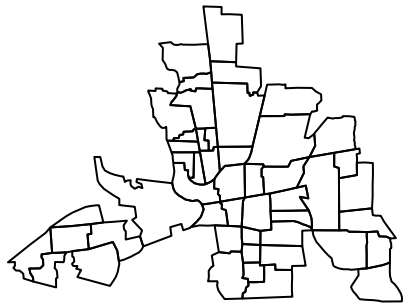
Geometry list/column

Single geometry

```
# basic data plots for overview
plot(columbus)
```




```
plot(st_geometry(columbus))
```



```
# check the projection (we already sw above that it's CRS WGS84,
# with this you can see also the ID "EPSG:4326")
st_crs(columbus)
```

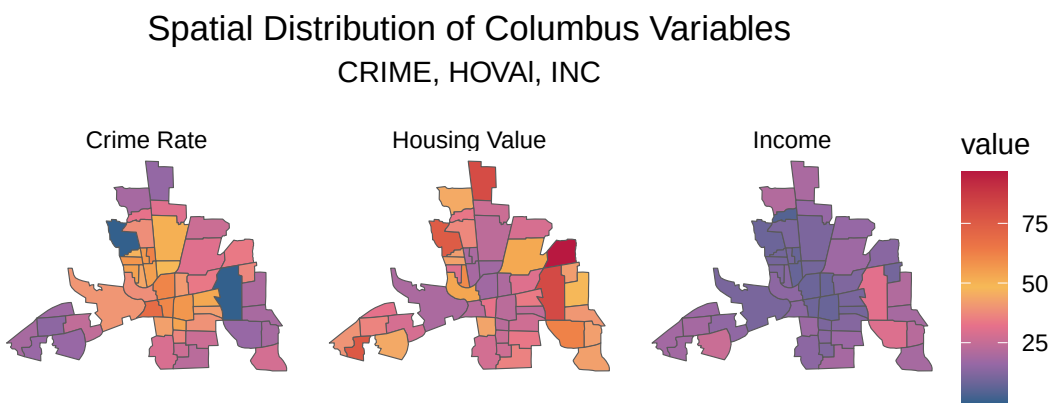
Plotting - Visual Inspection

Go to https://r-graph-gallery.com/ for graph examples!

Go to https://pmassicotte.github.io/paletteer_gallery/ for color palettes!

```
#some functions/ plotting methods (like facet_wrap(~ name) )
columbus_1 <- pivot_longer(columbus, cols = c(CRIME, INC, HOVAL))

#renamming the variables
columbus_1$name <- recode(columbus_1$name,
                            CRIME = "Crime Rate",
                            INC = "Income",
                            HOVAL = "Housing Value")

#plotting
ggplot(columbus_1) +
  geom_sf(aes(fill = value)) +
  theme_void() +

  # makes 3 plots, side by side, by variable
  facet_wrap(~ name) +

  # using a pretty colour pallete, using the above website
  scale_fill_paletteer_c("ggthemes::Sunset-Sunrise Diverging") +
```

```
  labs(title = "Spatial Distribution of Columbus Variables",
       subtitle = "CRIME, HOVAl, INC") +
  theme(plot.title = element_text(hjust = 0.5, margin = margin(b = 5)),
       plot.subtitle = element_text(hjust = 0.5, margin = margin(b = 15)))
```



Interpretation:

- **Crime Rate:** there is clear spatial clustering where neighborhoods with high crime are grouped together, suggesting some level of spatial dependency, indicating that crime in one area might be influenced by crime in neighboring areas.
- **Housing Value**: high value areas tend to be near other high value areas which points to a positive spatial autocorrelation.
- **Income**: this variable is relatively lower and more uniformely distributed but still the higher income areas are not randomly scattered which suggests again some level of spatial heterogeneity.

Therefore, there is indication of spatial autocorrelation and heterogeneity which is crucial as it tells us that if this dependence if ignored, OLS assumptions are violated. Here it would be best to use spatial models.

4.1.2 On Python

We can see the same on Python

```
gdf = gpd.read_file("columbus/columbus.shp")

#| fig-cap: "Distribution of Crime Rates"
sns.histplot(gdf['CRIME'], kde=True)
plt.title('Distribution of Crime Rates')
```

```
plt.xlabel('Crime Rate')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

#| fig-cap: "Choropleth Map of Crime by Neighborhood"
gdf.plot(column='CRIME', cmap='OrRd', legend=True, edgecolor='black')
plt.title('Crime Rate by Neighborhood')
plt.tight_layout()
plt.show()

#| fig-cap: "Choropleth Map of Income by Neighborhood"
gdf.plot(column='INC', cmap='YlGnBu', legend=True, edgecolor='black')
plt.title('Income by Neighborhood')
plt.tight_layout()
plt.show()

#| fig-cap: "Choropleth Map of Housing Value by Neighborhood"
gdf.plot(column='HOVAL', cmap='PuBuGn', legend=True, edgecolor='black')
plt.title('Housing Value by Neighborhood')
plt.tight_layout()
plt.show()
```

4.2 Spatial Dependence (spatial weights matrix)

Spatial dependence means that values in one area are influenced by values in nearby areas. To capture this, spatial models require a **"spatial weights matrix"**, i.e. a structure that defines who is "near" whom.

We know that the dataset is quite small (49 obs.) and that the geogrpahy is irregular (neighboorhoods are not all the same size of shape) so we use **K-Nearest Neighbors (KNN)** for our spatial weights.

k refers to the number of nearest neighbors that each spatial unit (in this case, each neighborhood) is connected to - for small datasets $k = 4$ is generally used.

- Eg. $k = 1$ would mean weak local influence; $k = 10$ would dilute spatial relationships

So, if $k = 4$, this means that each neighborhood will be linked to its 4 closest neighbors, based on the distance between their centroids (center points).

4.2.1 On R

First, we have to reproject the spatial data before doing KNN spatial weights to ensure the distances are appropriate. Our spatial data comes in geographic coordinates using

the WGS84 CRS (EPSG:4326), which is in degrees, not meters. So, calculating distances between points in degrees does not give meaningful results.

We choose to reproject to UTM (Universal Transverse Mercator) because it's one of the most reliable and practical projected CRS especially when calculating distances, areas, or nearest neighbors.

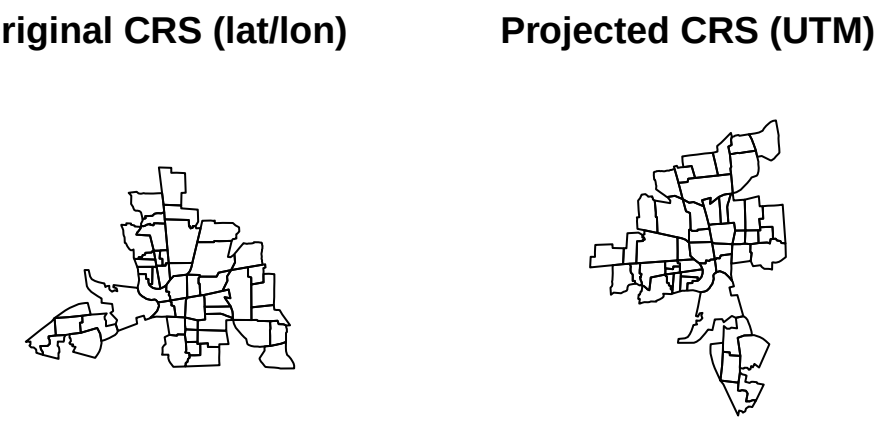
```
# above we had seen: https://spatialreference.org/
# reproject data to a projected CRS (EPSG:26917 = UTM )
columbus_proj <- st_transform(columbus, crs = 26917)

# also notice how the coordinates change
columbus$geometry
```

```
Geometry set for 49 features
Geometry type: POLYGON
Dimension:     XY
Bounding box:  xmin: 5.874907 ymin: 10.78863 xmax: 11.28742 ymax: 14.74245
Geodetic CRS:  WGS 84
First 5 geometries:
```

```
columbus_proj$geometry
```

```
Geometry set for 49 features
Geometry type: POLYGON
Dimension:     XY
Bounding box:  xmin: 13402530 ymin: 8407666 xmax: 15267820 ymax: 11220340
Projected CRS: NAD83 / UTM zone 17N
First 5 geometries:
```

```
#ploting the original CRS vs the projected CRS
par(mfrow = c(1, 2))
plot(st_geometry(columbus), main = "Original CRS (lat/lon)")
plot(st_geometry(columbus_proj), main = "Projected CRS (UTM)")
```



- **We can clearly see the changes in the map, which is not a rotation of the data, but due to the change in the projection.**

Now, we get the centroid (center point) of each polygon (neighborhood), so that we represent each neighborhood by a single point. Then we extract the coordinates (x, y) of those points to use for knn neighbor calculations.

```
# centroid coordinates
centroids <- st_centroid(columbus_proj)
coords <- st_coordinates(centroids)
```

Now, we identify, for each neighborhood (now represented by its centroid) we want to find the four closest neighbors based on geographic coordinates.

Then, we convert the neighbor object into a list of neighbors (nb) so now we have a structure that says, for each neighborhood, here are the indices of my 4 closest neighbors".

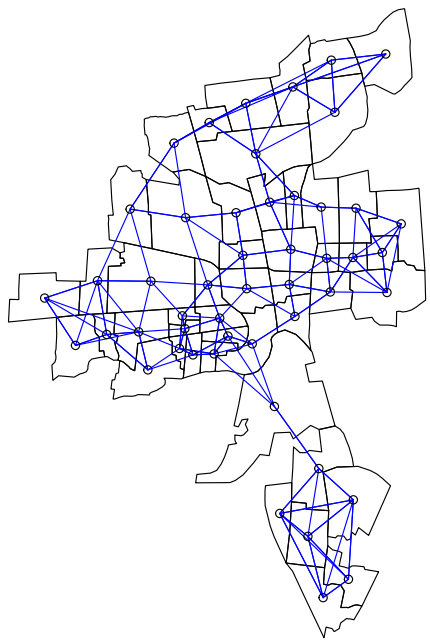
```
# 4-nearest neighbors
knn <- knearneigh(coords, k = 4)
nb <- knn2nb(knn)

# check the lenght -> you might have more coordinates than features
length(nb) == nrow(coords)
```

```
[1] TRUE
```

Now, we use the list of neighbors and turn it into a weighted neighborhoods structure (lw) (which is our spatial weights matrix.) The style "W" makes the weights row-standardized, meaning the weights for each observation sum to 1.

```r
# convert to spatial weights list
lw <- nb2listw(nb, style = "W")

# To see what it does, we can transform it into a matrix
W <- spdep::listw2mat(lw)
colnames(W) <- rownames(W) <- columbus_proj$name
view(W)

#in this dataset, the neighborhood names don't exist,
```

```r
# Plot what we have done so far
plot(st_geometry(columbus_proj))
plot(nb, coords, add = TRUE, col = "blue")
```



This network-style map can give a more intuitive sense of which districts are considered a "neighbor"in the model.

4.2.2 On Python

```python
# Reproject to a projected CRS to ensure centroids and distances are correct
gdf = gdf.to_crs(epsg=26917)
```

```python
# Calculate coordinates for KNN
coords = np.column_stack((gdf.geometry.centroid.x, gdf.geometry.centroid.y))
knn = KNN.from_array(coords, k=4)
knn.transform = 'R'

y = gdf['CRIME'].values.reshape(-1, 1)
X = gdf[['INC', 'HOVAL']].values


#| fig-cap: "Nonzero Structure of the Spatial Weights Matrix (W)"
import scipy.sparse

# Convert PySAL W object to a sparse matrix
W_sparse = knn.full()[0]
W_matrix = scipy.sparse.csr_matrix(W_sparse)

# Plot structure
plt.figure(figsize=(6,6))
plt.spy(W_matrix, markersize=1)
plt.title("Structure of Spatial Weights Matrix (W)")
plt.xlabel("To")
plt.ylabel("From")
plt.tight_layout()
plt.show()

{python}
#| fig-cap: "K-Nearest Neighbor Links between Neighborhoods"
from shapely.geometry import LineString

centroids = gdf.geometry.centroid
lines = []

for i, neighbors in knn.neighbors.items():
    for j in neighbors:
        lines.append(LineString([centroids.iloc[i], centroids.iloc[j]]))

links_gdf = gpd.GeoDataFrame(geometry=lines, crs=gdf.crs)

# Proper aspect ratio using custom Axes
fig, ax = plt.subplots(figsize=(8, 8))  # Square figure

gdf.plot(ax=ax, edgecolor='grey', facecolor='none')
links_gdf.plot(ax=ax, color='red', linewidth=0.8)

ax.set_title("Spatial Links Defined by W (4-Nearest Neighbors)")
ax.set_aspect('equal')  # Enforce 1:1 scaling of x and y axes
plt.tight_layout()
plt.show()
```

4.3 Estimating Spatial Models

We choose 4 models to compare how differently they interpret space:

Model	Spatial Dependency	Use	Model
OLS Regression	Does not account for spatial relations	Baseline for comparison	$y = \beta X + \varepsilon$
Spatial Lag Model (SAR)	Captures spatial dependence in the dependent variable Y	When diffusion or spillover effects are present (e.g., crime in 1 neighborhood affects nearby neighborhood).	$y = \rho W y + \beta X + \varepsilon$
Spatial Cross-Regressive Model (SLX)	Captures spatial dependence in the independent variable X	When spatially lagged predictors (e.g. income or housing value in neighboring areas) affect local crime.	$y = \beta X + WX\theta + \varepsilon$
Spatial Error Model (SEM)	Captures spatial dependence in the error term ε	When unmeasured spatial processes (e.g. missing variables like policing or social networks) influence crime patterns	$y = \beta X + u$; $y = \lambda W u + \varepsilon$

4.3.1 On R

Running the Models:

```
#OLS
ols_model <- lm(CRIME ~ INC + HOVAL, data = columbus_proj)
summary(ols_model)

# SAR
sar_model <- lagsarlm(CRIME ~ INC + HOVAL, data = columbus_proj, listw = lw)
summary(sar_model)$Coef

# SLX
slx_model <- lmSLX(CRIME ~ INC + HOVAL, data = columbus_proj, listw = lw)
summary(slx_model)$Coef

# SEM
sem_model <- errorsarlm(CRIME ~ INC + HOVAL, data = columbus_proj, listw = lw)
summary(sem_model)$Coef
```

Comparing the coefficients of the models:

```
#getting
ols_coefs <- coef(ols_model)
sar_coefs <- coef(sar_model)
```

```r
slx_coefs <- coef(slx_model)
sem_coefs <- coef(sem_model)

# extract spatial terms for SAR and SEM models
rho_sar <- sar_model$rho
lambda_sem <- sem_model$lambda

# for SLX, extract spatially lagged X terms (they're named like "lag.<var>")
wx_inc <- slx_coefs["lag.INC"]
wx_hoval <- slx_coefs["lag.HOVAL"]


# coefficient comparison table
coefs_df <- data.frame(
  Variable = c("Intercept", "INC", "HOVAL",
               "Spatial Lag (rho)", "Spatial Error (lambda)",
               "WX_INC", "WX_HOVAL"),
  OLS = c(ols_coefs["(Intercept)"], ols_coefs["INC"], ols_coefs["HOVAL"],
          NA, NA, NA, NA),
  SAR = c(sar_coefs["(Intercept)"], sar_coefs["INC"], sar_coefs["HOVAL"],
          rho_sar, NA, NA, NA),
  SLX = c(slx_coefs["(Intercept)"], slx_coefs["INC"], slx_coefs["HOVAL"],
          NA, NA, wx_inc, wx_hoval),
  SEM = c(sem_coefs["(Intercept)"], sem_coefs["INC"], sem_coefs["HOVAL"],
          NA, lambda_sem, NA, NA)
)

# round numeric values
coefs_df[ , 2:4] <- round(coefs_df[ , 2:4], 3)

# the table
print(coefs_df)
```

```
                Variable    OLS    SAR    SLX        SEM
1              Intercept 68.619 40.554 76.908 57.2943807
2                    INC -1.597 -0.950 -1.195 -1.0526335
3                  HOVAL -0.274 -0.257 -0.273 -0.2403171
4      Spatial Lag (rho)     NA  0.491     NA         NA
5 Spatial Error (lambda)     NA     NA     NA  0.6749742
6                 WX_INC     NA     NA -0.792         NA
7               WX_HOVAL     NA     NA -0.080         NA
```

Interpretation:

OLS: No spatial effects are included.

- A 1-unit increase in income reduces crime by **1.597 units**, holding other variables constant.
- A 1-unit increase in housing value reduces crime by **0.274 units**.

SAR: Adds spatial lag of the dependent variable (crime).

- ρ (rho) = **0.491:** Large and positive → suggests strong spatial autocorrelation in crime, i.e. if neighboring areas have higher crime, a given area tends to also have higher crime.
- Crime in a neighborhood is **partly explained** by crime levels in its 4 nearest neighbors.
- Income and housing still have a **negative** effect, but the total effect includes spatial spillover, meaning some of the explanatory power that OLS attributes to income/housing is now explained by spatial contagion.

SLX: This model includes spatially lagged independent variables.

- Crime is influenced not just by local income and housing value, but also by the average values in the 4 nearest neighborhoods.
  - **WX_INC = -0.792**: If neighboring areas have higher income, the crime in the focal area tends to be lower.
  - **WX_HOVAL = -0.080**: A smaller but still negative effect of neighboring housing values.

Suggests contextual spillovers, i.e. wealthier neighbors correlate with lower crime in adjacent areas.

SEM: Captures spatial correlation in the error term, rather than in the dependent variable.

- λ(lambda) = **0.675**: strong spatial autocorrelation in unobserved factors (e.g., policing, infrastructure, social networks).
  - The observed variables explain part of crime, but unmeasured spatial processes explain another large portion.

4.3.2 On Python

```
# Fit OLS model
ols_model = OLS(y, X, name_y='CRIME', name_x=['INC', 'HOVAL'])

# Fit Spatial Lag model
lag_model = ML_Lag(y, X, w=knn, name_y='CRIME', name_x=['INC', 'HOVAL'])
```

4.4 Spatial AutoCorrelation

Spatial autocorrelation tells us how much a variable is spatially patterned, i.e., whether similar values tend to cluster in space. It helps us determine whether spatial data values are clustered, dispersed, or randomly distributed.

- If high-crime areas are near other high-crime areas, that's **positive spatial autocorrelation**.
- If high-crime areas are surrounded by low-crime areas, that's **negative spatial autocorrelation**.
- If the spatial pattern is random, there's **no autocorrelation**.

To measure spatial autocorrelation, we use **Global Moran's I** and **Local Moran's I (LISA)**:

- **Global Moran's I :** measures the overall spatial autocorrelation across the entire dataset
  - Moran's I +1 : strong positive spatial autocorrelation (similar values cluster together, high-high or low-low)
  - Moran's I 0 : no spatial autocorrelation (random spatial pattern)
  - Moran's I -1 : strong negative spatial autocorrelation (high next to low)
    - **p < 0.05**: Moran's I is statistically significant
    - **p 0.05**: Moran's I is not statistically significant.
  - Most used for diagnostics

- **Local Moran's I (LISA):** measures spatial autocorrelation at each location
  - LISA Cluster = High-High (Hot Spot)
  - LISA Cluster = Low-Low (Cold Spot)
  - LISA Cluster = High-Low or Low-High (Outlier)

4.4.1 On R

Global Moran's I

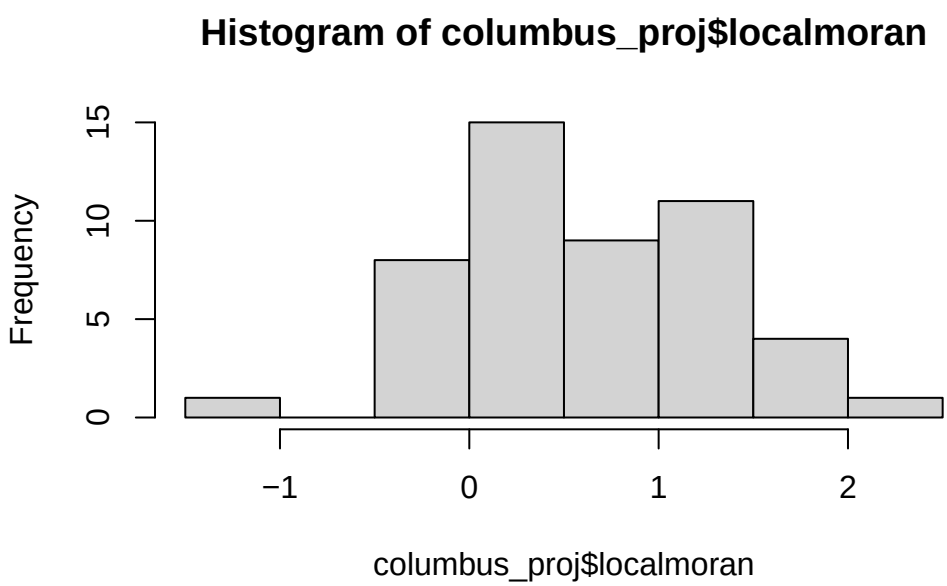
```r
# global autocorrelation in the raw data
moran.test(columbus_proj$CRIME, lw)
```

```
    Moran I test under randomisation

data:  columbus_proj$CRIME
weights: lw

Moran I statistic standard deviate = 6.8191, p-value = 4.582e-12
```

```
alternative hypothesis: greater
sample estimates:
Moran I statistic       Expectation          Variance
      0.599493770      -0.020833333       0.008275444
```

Local Moran's I

```r
# local autocorrelation
loc <- localmoran(columbus_proj$CRIME, lw)
loc <- as.data.frame(loc)

# extract local moran's I
columbus_proj$localmoran <- loc$Ii

# extract p-values for significance
columbus_proj$pmoran <- loc$`Pr(z != E(Ii))`

# histogram
hist(columbus_proj$localmoran)
```



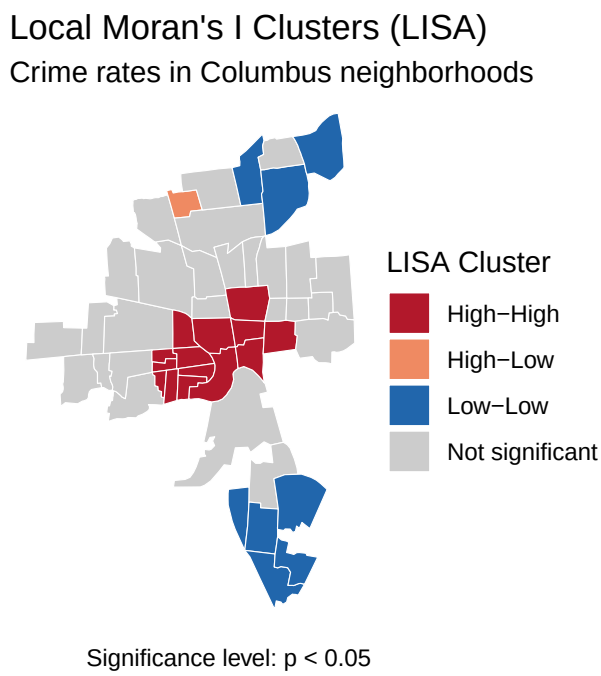
Interpretation:

Most values are concentrated between 0 and 2, suggesting positive local spatial autocorrelation , i.e. neighborhoods with similar crime levels (either high or low) tend to be near each other.

A few values are below 0, indicating some negative autocorrelation, either potential outliers or boundaries between high and low crime areas.

Finding Statistically Significant Clusters

```r
# column to identify areas with significant local autocorrelation
columbus_proj$sig_cluster <- columbus_proj$pmoran < 0.05

#classifying cluster types
mean_crime <- mean(columbus_proj$CRIME)
crime_centered <- columbus_proj$CRIME - mean_crime
lagged_crime <- lag.listw(lw, crime_centered)

columbus_proj$LISA_type <- NA
columbus_proj$LISA_type[crime_centered > 0 &
                        lagged_crime > 0 &
                        columbus_proj$pmoran < 0.05] <- "High-High"
columbus_proj$LISA_type[crime_centered < 0 &
                        lagged_crime < 0 &
                        columbus_proj$pmoran < 0.05] <- "Low-Low"
columbus_proj$LISA_type[crime_centered > 0 &
                        lagged_crime < 0 &
                        columbus_proj$pmoran < 0.05] <- "High-Low"
columbus_proj$LISA_type[crime_centered < 0 &
                        lagged_crime > 0 &
                        columbus_proj$pmoran < 0.05] <- "Low-High"


#plotting

lisa_colors <- c(
  "High-High" = "#b2182b",
  "Low-Low" = "#2166ac",
  "High-Low" = "#ef8a62",
  "Low-High" = "#67a9cf",
  "Not significant" = "grey80"
)

# Ensure 'LISA_type' has a default for non-significant areas
columbus_proj$LISA_type <- ifelse(is.na(columbus_proj$LISA_type),
                                  "Not significant",
                                  columbus_proj$LISA_type)

# Plot
ggplot(data = columbus_proj) +
  geom_sf(aes(fill = LISA_type), color = "white", size = 0.3) +
  scale_fill_manual(values = lisa_colors, name = "LISA Cluster") +
  theme_void() +
  labs(title = "Local Moran's I Clusters (LISA)",
       subtitle = "Crime rates in Columbus neighborhoods",
       caption = "Significance level: p < 0.05") +
  theme(legend.position = "right")
```



Model Fit

AICc is a statistical metric used to compare models and choose the one that balances goodness of fit and model complexity, especially when there is small sample sizes. Lower AICc = better model (values between 0-2; >10 is unlikely to be best model). We use Δ AICc to show how much worse a model is compared to the best one.

```
# AICc function
aicc <- function(model) {
  k <- length(coef(model))
  n <- length(residuals(model))
  aic_val <- AIC(model)
  aic_val + (2 * k * (k + 1)) / (n - k - 1)}

# models
model_list <- list(
  OLS = ols_model,
  SAR = sar_model,
  SEM = sem_model,
  SLX = slx_model)

# AICc data frame
aicc_df <- data.frame(
  Model = names(model_list),
  AICc = sapply(model_list, aicc))

# Delta_AICc and Rank
aicc_df <- aicc_df %>%
  arrange(AICc) %>%
  mutate(
    Delta_AICc = round(AICc - min(AICc), 3),
```

```r
    Rank = rank(AICc, ties.method = "min"))

# Moran's I and p-value
get_moran <- function(model, lw) {
  res <- residuals(model)
  moran_res <- moran.test(res, lw)
  c(Moran_I = round(unname(moran_res$estimate["Moran I statistic"]), 4),
    Moran_p_value = round(moran_res$p.value, 4))}

# Moran's I test to all models
moran_matrix <- t(sapply(model_list, get_moran, lw = lw))
moran_df <- as.data.frame(moran_matrix)
moran_df$Model <- rownames(moran_df)
rownames(moran_df) <- NULL

final_results <- left_join(aicc_df, moran_df, by = "Model")
print(final_results)
```

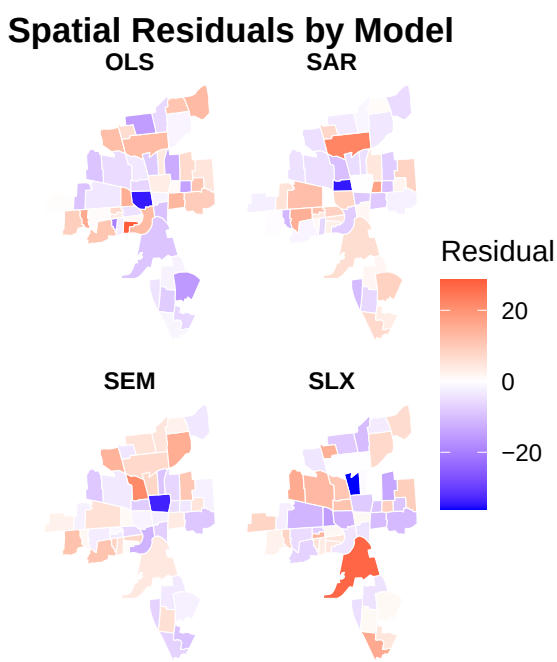
```
  Model     AICc Delta_AICc Rank Moran_I Moran_p_value
1   SEM 368.4360      0.000    1  0.0218        0.3122
2   SAR 368.5320      0.096    2  0.0855        0.1110
3   SLX 383.1778     14.742    3  0.3541        0.0000
4   OLS 383.2878     14.852    4  0.3736        0.0000
```

The **SEM** model most effectively addresses spatial autocorrelation, followed by **SAR**.

Plotting Residuals

```r
#Plotting Residuals

# Ensure residuals are attached
columbus_proj$res_ols <- residuals(ols_model)
columbus_proj$res_sar <- residuals(sar_model)
columbus_proj$res_slx <- residuals(slx_model)
columbus_proj$res_sem <- residuals(sem_model)

# Extract residual columns manually (bypass `select()`)
res_df <- data.frame(
  res_ols = columbus_proj$res_ols,
  res_sar = columbus_proj$res_sar,
  res_slx = columbus_proj$res_slx,
  res_sem = columbus_proj$res_sem
)

# Pivot to long format
res_long <- pivot_longer(res_df, cols = everything(),
                         names_to = "Model", values_to = "Residual")
```

```r
# Repeat geometry to match long format
geom_rep <- st_geometry(columbus_proj)
res_long$geometry <- rep(geom_rep, times = 4)
res_long <- st_sf(res_long)

# Rename models
res_long$Model <- recode(res_long$Model,
                         res_ols = "OLS",
                         res_sar = "SAR",
                         res_slx = "SLX",
                         res_sem = "SEM")

# Plot
ggplot(res_long) +
  geom_sf(aes(fill = Residual), color = "white", size = 0.2) +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red") +
  facet_wrap(~ Model, ncol = 2) +
  theme_void() +
  labs(
    title = "Spatial Residuals by Model",
    fill = "Residual"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    strip.text = element_text(face = "bold")
  )
```



Interpretation:

OLS: Strong residuals in both directions (deep red and blue patches), indicating **spatial autocorrelation** is present and unaccounted for.

SAR: Shows some spatial smoothing: neighborhoods with high or low residuals tend to be surrounded by similar values.

SEM: Overall lowest magnitude of residuals, the residuals appear more balanced and spatially random. This suggests the **spatial error model has best fit.**

SLX: Some very strong residuals (especially one dark red neighborhood). Better than OLS worse than SAR & SEM.

This suggests that OLS and SLX have heteroskedasticity, their residuals vary too much across space. SEMreduces heteroskedasticity because it better models unobserved spatial processes, but it does not eliminate it completely.

These residual maps strongly reinforce the numeric findings!

4.4.2 On Python

```
moran_resid = Moran(ols_model.u, knn)
print(f"Moran's I: {moran_resid.I}, p-value: {moran_resid.p_sim}")

#| fig-cap: "Spatial Distribution of OLS Residuals"
gdf['OLS_resid'] = ols_model.u
gdf.plot(column='OLS_resid', cmap='coolwarm', legend=True, edgecolor='black')
plt.title('OLS Residuals by Neighborhood')
plt.tight_layout()
plt.show()
```

5 Main Conclusions

1. **Crime is Spatially Dependent**

- Crime rates in Columbus neighborhoods are not randomly distributed. There is clear spatial autocorrelation:
  - High-crime areas are near other high-crime areas.
  - Same applies to income and housing values.

2. **Socioeconomic factors matter but don't explain everything:**

- Higher income and housing values are associated with lower crime. However, this is not the full story: neighboring values (lag variables) also affect outcomes (seen in SAR and SLX)

3. **Spatial Models help improve our understanding:**

- 4 models were compared and based on AICc, residual plots, and Moran's I, SEM provided the best statistical fit, least spatial autocorrelation in residuals, and a **strong indication that unobserved spatial processes (like policing, infrastructure, etc.) influence crime patterns.**

6 Final Reflections

6.1 Limitations

1. **The data is from 1980** — over 45 years old. Urban patterns, demographics, crime trends, and housing values have changed dramatically since then.
2. Only 49 neighborhoods in 1 city (Columbus), while suitable for methodological demonstration, it limits generalizability and statistical power.
3. **Crime data is not neutral:** it reflects not only actual events but also biased policing, racialized surveillance, and systemic underreporting of certain harms (e.g., gender-based violence). Moreover, by focusing narrowly on socioeconomic indicators like income and housing value, we risk obscuring deeper structural factors like historical segregation, gentrification, and education access. Without a broader social lens, spatial models may unintentionally reinforce the very inequalities we aim to study.
4. No sensitivity analysis is done to assess how results change with $k = 3$ or $k = 5$.

7 Additional Resources (Data Sets, R & Python)

Using spatial data with R: https://cengel.github.io/R-spatial/

Geocomputation with R: https://r.geocompx.org/

Online course: https://www.earthdatascience.org/courses/earth-analytics/spatial-data-r/

Nice set of Youtube Tutorials: Geospatial Data with R

A book on spatial data with R: https://asdar-book.org/

For those who want to understand Python: https://www.w3schools.com/python/

Working with Geopandas on Python: https://geopandas.org/en/stable/

Python Geospatial Libraries: https://github.com/SpatialPython/spatial_python/blob/master/packages.md

---

Some interesting spatial databases to look into for exercises:

→ Mobility Database

→ Open StreetMap

→ US Census Spatial Data

→ EU Spatial Data

→ World Pop

→ Open Street Map

→ Humanitarian Data Exchange